

EngineTwin Ground Station

How the digital twin works, what every control on the console does, and how to drive it — from a cold start to a post-flight debrief.

PROBLEM STATEMENT

SIH26054 · DRDO

ENVELOPE

0 - 30,000 ft

ACCESS

http://127.0.0.1:8000

POWERPLANT

VRDE 2.2 L turbocharged aero-diesel

UPDATE RATE

20 Hz · 1.5 ms of a 150 ms budget

PROVENANCE

All telemetry SIMULATED

00 Contents

02	Starting it up	install, launch, first screen
03	The cover journey	22 chapters, then Continue
04	Console layout	what is where, and why
05	Reading the graphs	all three charts, line by line
06	The 3D engine model	colouring, picking, inspector
07	Every control, explained	complete reference
08	Predicting faults	how a diagnosis is produced
09	Manual flight sandbox	flying it yourself
10	Recording, debrief and replay	the flight data recorder
11	Demonstration scripts	what to show, in what order
12	The HTTP and WebSocket API	driving it from code
13	Verifying the claims	reproducing every number
14	Troubleshooting	when something looks wrong
15	Known limitations	stated, not buried

01 What the system actually is

A conventional engine instrument system watches each channel against a fixed redline. On a light aircraft at 8,000 ft that works. On a MALE UAV climbing to 30,000 ft on a twenty-hour sortie it fails in *both* directions at once.

The two failures of a fixed threshold

Climb high enough and a **healthy** engine starts tripping alarms. Manifold pressure falls as the turbocharger runs out of air, rated power derates from 200 HP to 110 HP, exhaust temperatures drift. Nothing is broken — the atmosphere changed. The operator learns to ignore the alarm.

Meanwhile a genuinely **failing** engine sits comfortably inside every redline. An injector clogging on cylinder 3 pushes EGT₃ up by 40 °C, well under the 880 °C continuous limit, so nothing trips — right up until it does.

The trap

A raw sensor reading carries two things at once: the state of the engine, and the state of the flight. You cannot threshold your way out of that, and a black-box model trained on raw telemetry inherits the same problem — it just hides it behind a confidence score.

What EngineTwin does instead

A Mean Value Engine Model (MVEM) runs in lockstep with the engine, fed the same altitude, ambient conditions and throttle. It predicts what a *healthy* engine would be doing right now. Diagnosis then happens on the difference, normalised per channel:

$$r_i = (y_{\text{measured}} - y_{\text{physics}}) / \sigma_{\text{channel}}$$

Fifteen channels — RPM, manifold pressure, fuel flow, oil pressure and temperature, four CHT, four EGT, vibration RMS and bus voltage — reduce to one vector of standard deviations from thermodynamic expectation.

Because the physics baseline has already absorbed altitude, throttle and ambient temperature, a healthy engine sits at $r \approx 0$ everywhere in the envelope. The learning problem stops being “recognise this engine” and becomes “recognise a departure from physics” — a far smaller and far more transferable problem.

The one sentence to remember

Every model downstream sees residuals. None of them ever sees a raw sensor value. That single decision is what the whole system rests on.

What it buys, measured

METRIC	FIXED-THRESHOLD EIS	BLACK-BOX ML	ENGINETWIN
Classification accuracy	53.65 %	86.93 %	97.64 %
Macro F1	0.2463	0.8696	0.9770
Mean detection latency	28.26 s	8.00 s	2.87 s
False alarms on healthy sorties	0.00 %	11.74 %	1.42 %

48,000 held-out frames from 32 independent sorties. Reproduce with `python validation/benchmark.py`.

Read the EIS column carefully. A perfect 0.00 % false-alarm rate beside a macro F1 of 0.25 is not a strength — the thresholds are set so wide that almost nothing real trips them.

02 Starting it up

Requirements

Python 3.10 or 3.11, and a browser with WebGL. Nothing else.

First run

- 1 Create an environment and install.

```
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # macOS / Linux
pip install -r requirements.txt
```

- 2 **Launch.** `python main.py` , or double-click `run_demo.bat` on Windows. The launcher opens `http://127.0.0.1:8000` automatically.
- 3 **Wait for warm-up.** The twin runs forward 15 seconds before accepting clients, so the cylinder thermal states and the estimator have settled. Without it the first frame would read CRITICAL on a perfectly healthy engine.
- 4 **You land on the cover.** Scroll down through it, or press `End` to jump to the Continue button.

No internet required

Tailwind, Chart.js, Three.js and the webfont are all vendored under `dashboard/vendor/` and served from the twin's own host. The station comes up on a closed network. Refresh those copies only if you need to, with `python tools/fetch_vendor_assets.py` .

If FastAPI is missing

`main.py` falls back to `standalone_server.py` , which serves the whole twin on the Python standard library alone. Telemetry then arrives over Server-Sent Events instead of a WebSocket; the dashboard switches transports by itself and nothing else changes. NumPy and PyTorch are still required for the physics and the networks.

Where things live

PATH	CONTAINS
simulation/	MVEM physics, sensor model, flight profiles, fault injector
digital_twin/	State estimator, health index, twin state, flight recorder
models/	Autoencoder, classifier, sensor validator, RUL estimator
xai/	Attribution and operator alert generation
dashboard/	The ground station: cover, console, 3D view, theme
validation/	Benchmark, ablation, leakage audit, stress tests
data/flight_logs/	Recorded sorties (created at run time, not in git)

03 The cover journey

The cover is not a separate page. It and the console are two fixed layers of one document, and scrolling drives the handover between them. Twenty-two chapters carry the argument from the problem through the evidence to the limitations.

Navigating it

Scroll wheel, trackpad, touch	Advances the journey. The rendered position eases toward the scroll position rather than snapping to it, which is what makes a wheel notch glide instead of step.
Altitude rail left edge	Reads 0 to 30,000 ft across the journey. It is the one piece of chrome that never leaves, and it ties the descent through the page to the climb through the envelope.
CONTINUE final chapter	Hands over to the console. The cover scales past you and blurs out while the console un-blurs from behind it, over 750 ms.
↑ Cover console header	Returns to the journey. Only visible once the console is live.

Why the page stops scrolling after Continue

Once the console is live the scroll track collapses to zero height. Both layers are fixed, so nothing moves on screen — the page simply stops having anywhere to scroll. Without this, a stray wheel gesture over a panel would drag you back out to the cover in the middle of a fault.

The twenty-two chapters

1. Hero and masthead
2. Why threshold alarms fail at 28,000 ft
3. A reading carries two things at once
4. The residual equation
5. What happens in one 50 ms tick
6. One continuous climb, measured
7. Built for mission-critical operation
8. Performance at a glance
9. Six layers, one diagnostic chain
10. The five levels of the twin
11. What the MVEM integrates
12. The instrument model and its sigmas
13. The ten-class fault library
14. A dead probe is not a dead cylinder
15. Benchmark against the alternatives
16. The ablation: what is load-bearing
17. Per-class precision and recall
18. Seventy-eight independent missions
19. The 1.5 ms compute budget
20. What this is not

21. Deployment roadmap

22. Continue to the console

04 Console layout

```
+-----+
| STATUS BAR  identity | mission tape | station controls |
+-----+
| [ SANDBOX ] collapsible - manual flight controls |
| [ DEBRIEF ] collapsible - recorded sorties and replay |
+-----+
| 3D ENGINE MODEL | EXHAUST & HEAD | DIAGNOSIS |
| cylinder strip | TEMPERATURE | condition, class, |
| component inspector | confidence, severity |
+-----+
| SUBSYSTEM HEALTH | OIL PRESSURE & | ROOT-CAUSE |
| 7 scores | MANIFOLD BOOST | ATTRIBUTION |
+-----+
| RECOMMENDED ACTION | VIBRATION & POWER | EVENT LOG |
+-----+
| FAULT INJECTION | 8 one-click failure modes | auto demo | rate |
+-----+
```

The three columns

Left is the engine as an object: the 3D model, per-cylinder temperatures, subsystem health and the directive. **Centre** is the engine as signals over time: three charts, each showing measurement against physics. **Right** is the engine as a conclusion: what the AI thinks, why it thinks it, and what happened when.

The status bar

Left is identity — station name, problem statement, powerplant. Centre is the mission tape: mission time, flight phase, altitude, true airspeed, outside air temperature and throttle. Right is the station controls and the telemetry indicator.

Annunciation doctrine

Colour is never decoration on this console. Structure is neutral; only a parameter out of tolerance is allowed to be saturated.

COLOUR	MEANS	WHERE IT MAY APPEAR
 VIOLET	What physics says	Model-expected traces, 3D selection outline, inspector model column, focus rings
 AMBER	Out of tolerance	Caution band, panel left border, readout, log tag
 RED	Hard limit exceeded	Warning band, panel left border, readout
 GREEN	Healthy / live	Health badge and the telemetry-live indicator only
Neutral	Structure	Everything else: panels, rules, axes, inactive controls, nominal readouts

Reading the panel borders

A panel signals an abnormal state with a coloured left border and nothing else — never a coloured background, never a glow. If a panel edge is amber, something inside it left tolerance.

05

Reading the graphs

All three charts share one convention: **solid lines are what the sensors measured; dashed violet is what the physics model expected.** The distance between them is the residual, and the residual is the entire diagnosis. If solid and dashed sit on top of each other, the engine is doing what physics says it should — whatever the absolute numbers are.

Chart 1 — Exhaust & head temperature

SERIES	STYLE	MEANING
EGT 1–4	Four solid neutral traces, lightest to darkest	Exhaust gas temperature per cylinder. Told apart by lightness, never hue.
Model expected	Dashed violet	What the MVEM says EGT should be right now.

A trace turns amber above 880 °C and red above 950 °C, and the chart carries tinted bands at those levels. A single EGT climbing away from its three siblings is the signature of a mixture fault on that cylinder. All four rising together is usually the flight condition, not a fault — which is exactly what the dashed line will confirm.

Chart 2 — Oil pressure & manifold boost

SERIES	AXIS	MEANING
Oil pressure	Left	Lubrication circuit pressure, bar. A <i>falling</i> parameter: low is the fault direction.
Oil P expected	Left, dashed violet	Model expectation for the same instant.
Manifold MAP	Right	Manifold absolute pressure, bar. Falls with altitude by design.
MAP expected	Right, dashed violet	Model expectation, which falls with it.

The altitude test

Watch MAP during a climb. The solid line falls steeply — and the dashed line falls with it, so the residual stays flat and no alarm fires. That is the entire argument of the project, visible in one chart.

Chart 3 — Order-tracked vibration & power

SERIES	AXIS	MEANING
Vibration 2X RMS	Left	Vibration at twice crankshaft order, g RMS. Amber at 2.5 g, red at 4.0 g.
Power	Right, dashed violet	Model shaft power, HP.

Bearing wear shows as slow growth in the 2X order, not as a step. Watching this trace rise over a minute while power holds steady is the classic developing-mechanical-fault picture.

Root-cause attribution — horizontal bars

Not a time series. When a fault is named, this panel ranks the top five residual channels that drove the decision, with the percentage each contributed and how many sigma it sits from expectation. Bars turn amber or red as that channel's own residual leaves tolerance.

What this is, precisely

Gradient × input attribution, not Shapley values. It gives a defensible *ranking* of which channels moved the decision, not an additive decomposition with SHAP’s guarantees. It was chosen because it costs one backward pass and fits a 50 ms edge budget.

06 The 3D engine model

The model is an instrument, not decoration. It is a procedural VRDE 2.2 L four-cylinder turbocharged aero-diesel, and everything it shows is driven by the live twin state.

How components are coloured

Each cylinder is shaded by its thermal deviation *from the model*, measured in sigma — not by absolute temperature. A cold-day 700 °C and a hot-day 760 °C both read as nominal; only a genuine departure changes colour.

DEVIATION	STATE	APPEARANCE
< 2.0 σ	NOMINAL	Neutral metal
2.0 - 4.0 σ	CAUTION	Amber
≥ 4.0 σ	WARNING	Red

The oil sump, turbocharger and crankcase are coloured by their subsystem health score instead, going amber below 85 % and red below 50 %. When a fault is active, the implicated component is outlined and the outline breathes slowly so it reads as a live annunciation.

Interacting with it

Drag left mouse	Orbits the camera. Dragging never changes the selection — a click is distinguished from the end of a drag.
Scroll over model wheel	Zooms, clamped between 1.6 and 16 units.
Hover pointer	Lifts the component under the cursor with a violet emissive tint and switches the cursor to a pointer.
Click pointer	Selects the component, frames the camera on it, outlines it in violet and opens the inspector. Clicking the same part again, or empty space, clears the selection.
Esc keyboard	Clears the selection.

Selectable components

Cylinders 1–4, the four EGT probes, the turbocharger, the oil sump, the crankcase and the propeller. **Probes are separate components from their cylinders** — a dead thermocouple and a dead cylinder are different findings, so the model lets you point at each independently.

The component inspector

Selecting a part opens a readout beneath the model with one row per channel:

COLUMN	SHOWS
Channel	Which measurement this row describes
Measured	What the sensor read, in engineering units
Model	What physics expected — in violet, matching the dashed chart traces
Residual	The gap between them in sigma, amber past 3σ , red past 6σ

The model, the cylinder strip and the subsystem health tiles are three views of one selection — selecting in any of them selects in all of them.

07

Every control, explained

Status bar

Sandbox ▾	Opens the manual flight panel. See section 09.
Debrief ▾	Opens the post-flight panel. See section 10.
↑ Cover	Returns to the cover journey. Only shown once the console is live.
Brief ↗	Opens the standalone project brief in a new tab.
Audio: muted	Toggles aural alerts. Off by default. When on, a two-tone alert sounds as an annunciation escalates.
Telemetry live • 20 Hz	Not a control — a status indicator. It names the transport in use: WS (WebSocket), SSE or HTTP polling. The dashboard falls back automatically in that order.
Reset	Restarts the simulation from $t = 0$, clears any active fault, reseeds the sensor noise and re-runs the 15-second warm-up. Also clears the RUL history and the event log buffers.

Engine model panel

Orbit / Cyls / Turbo / Sump	Camera presets: full view, cylinder bank, turbocharger, oil sump. The camera eases to the new position rather than cutting.
Labels	Shows or hides the component name sprites (CYL 1–4, TURBO, SUMP, PROP).
Cylinder strip four cells below the model	Per-cylinder EGT and the deviation from expectation. Clicking a cell selects that cylinder in the model.
✕ inspector	Clears the component selection and closes the inspector.

Subsystem health tiles

Seven scores, each $100 \cdot \exp(-\alpha \cdot |r|)$ over the relevant residuals, banded

NOMINAL

 85–100, ADVISORY 70–85,

CAUTION

 50–70,

WARNING

 25–50 and CRITICAL below 25.

TITLE	COVERS	CLICKABLE
Overall	Weighted composite of all subsystems	—
Oil system	Oil pressure and temperature residuals	Selects the sump
Turbo / boost	Manifold pressure residual	Selects the turbocharger
Combustion	Four EGT and four CHT residuals	Selects cylinder 1
Vibration 2X	Vibration RMS residual	Selects the crankcase
Avionics bus	Bus voltage residual	—

Fault injection strip

Eight one-click failure modes. Each injects its fault at the current simulation time with a default severity of 1.0 and a short ramp. The active button takes a coloured left border matching the severity of the fault.

BUTTON	INJECTS	EXPECT TO SEE
Healthy baseline	HEALTHY	Clears all faults; health returns to ~100 %
Oil pressure loss	OIL_PRESSURE_LOSS	Oil pressure below 2 bar in under 3 s; sump red; RTB directive
Probe defect (EGT3)	SENSOR_FAULT_EGT3	EGT3 falls to ambient; health barely moves; sensor-fault verdict
Bearing 2X vib	BEARING_WEAR_VIBRATION	2X vibration grows; crankcase amber; inspection window in hours
Turbo boost loss	TURBO_BOOST_DEFICIENCY	MAP deficit that deepens with altitude
Lean mixture cyl 3	LEAN_MIXTURE_CYL3	EGT3 climbs away from its siblings
Cooling degradation	COOLING_DEGRADATION_CYL2	CHT2 rises fast, EGT largely unaffected
Cylinder misfire	CYLINDER_MISFIRE_TIMING	RPM instability and a 0.5× vibration order

Simulation rate

1x · 2x · 5x

Time scale. Multiplies how much simulated time passes per real second, clamped server-side between 0.2× and 10×. Higher rates make slow-developing faults such as bearing wear practical to demonstrate. The manual sandbox slews in simulation time, so these also speed up a commanded climb.

Auto demo

► Auto demo

Runs a scripted six-phase arc unattended, 40 seconds total. Press again, or **Stop**, to end it early.

PHASE	TITLE	DURATION	WHAT HAPPENS
1	Healthy baseline	8 s	Model tracks every channel inside $\pm 1\sigma$
2	Fault injection	5 s	Oil pressure loss injected on a 1.0 s ramp
3	Anomaly detection	6 s	Reconstruction error crosses the calibrated threshold
4	Diagnosis	7 s	Classifier isolates the root cause; attribution ranks the evidence
5	RUL collapse	7 s	Prognostic horizon revised down; superseded range retained on screen
6	Recovery	7 s	Fault cleared; health and prognosis return to baseline

08 Predicting faults — how a diagnosis is produced

Twenty times a second the service runs the following chain.

Understanding the order matters, because each stage is allowed to stop the ones after it.

```
flight profile → MVEM integrate → apply faults → sensors sample
|
+--> estimator predict/update → residuals r = (y - y_hat)/sigma
|
+--> health indices (always)
|
+--> anomaly trigger --(below threshold)→ tick ends
|
+--> sensor validator --(probe fault)→ report and stop
|
+--> classifier → severity
|
+--> attribution + RUL → operator card
```

Stage by stage

- 1 **Flight profile advances.** ISA atmosphere gives ambient pressure and temperature for the current altitude; the mission phase sets throttle. Phases run TAXI_WARMUP, CLIMB_TO_ALTITUDE, HIGH_ALT_LOITER, DESCENT, APPROACH_LANDING.
- 2 **MVEM integrates.** Manifold filling, turbo boost with altitude derating, crankshaft torque balance, four cylinder thermal states, the oil circuit. This is ground truth.
- 3 **Faults are applied,** if any are active, as parametric perturbations of the physics or the instrumentation.
- 4 **Sensors sample** that truth through per-channel noise, first-order lag, quantisation and sample rate.
- 5 **The estimator** fuses measurement and physics into a 12-state estimate. If the datalink drops, it coasts on physics instead of producing a spurious residual spike.
- 6 **Residuals are computed** against a *second* MVEM that never saw the fault, and normalised per channel.
- 7 **Health indices** map residuals to bounded 0–100 % scores. This happens every tick regardless of what follows.
- 8 **Anomaly trigger.** An autoencoder trained only on healthy residuals reconstructs the vector. If the error clears mean + 3σ of the healthy calibration distribution, the rest of the stack wakes up. Otherwise the tick ends here — which is why the classifier only runs on about 1.2 % of frames.
- 9 **Sensor validator** decides, before anything else, whether this is a bad probe or a bad engine.
- 10 **Classifier** returns a calibrated fault class and a continuous severity.
- 11 **Attribution and RUL** attach the evidence and the remaining-life interval.

Why the trigger comes before the classifier

The autoencoder is unsupervised — it flags *anything* unlike healthy flight, including failure modes absent from the taxonomy. Gating the classifier behind it also saves 52 % of inference CPU, because during nominal flight the expensive stage never runs.

Reading a diagnosis

FIELD	HOW TO READ IT
Condition	The named fault class, or HEALTHY. Coloured by severity.
Confidence	Temperature-scaled, so it means something. An uncalibrated network reports 99 % on marginal cases; this one does not.
Severity	Continuous 0–1 from a separate regression head, independent of class confidence.
Anomaly index	0–1, derived from reconstruction error against the calibrated threshold.
RUL window	Remaining life in flight hours, as a range. See the caveat below.
Sensor fault flag	True when the validator attributes the anomaly to instrumentation rather than the engine.

The RUL number is a trend interval, not a calibrated one

It is built from the spread of the degradation-trajectory fit, not from a distribution-free coverage guarantee. Do not describe it as a 95 % confidence interval. When health has already fallen below the 30 % condemnation threshold it reports zero, because reporting hours remaining on an engine that is already past its limit — while the banner says return to base — would leave an operator believing neither number.

Predicting rather than reacting

To watch prediction rather than detection, inject a *slow* fault and leave it running: bearing wear at 5× rate is the clearest. The 2X vibration trace climbs gradually, overall health drifts down, and the RUL window narrows step by step. The previous estimate stays on screen beneath the current one, so the revision itself becomes evidence that the trend is real.

09 Manual flight sandbox

The sandbox lets you fly the aircraft yourself instead of following the scripted mission. It is how you demonstrate the central claim: that a healthy engine stays healthy across the whole envelope.

Controls

Mode: auto mission toggle	Switches between the scripted profile and manual control. In manual, the three sliders command the aircraft.
Throttle 0 - 100 %	Commanded throttle. Slews at 25 %/s.
Altitude 0 - 35,000 ft	Commanded altitude. Slews at 3,000 ft/min. The ISA atmosphere is recomputed at the commanded altitude, so the turbocharger genuinely sees thinner air.
OAT -50 - +45 °C	Outside air temperature. Slews at 6 °C/s.

The sliders are setpoints, not teleports

Commanded values are slewed at rates the airframe can actually fly. An instantaneous altitude step is not a manoeuvre — it is a discontinuity, and because the sensors carry first-order lag while the model baseline does not, a step opens a residual gap the classifier correctly reads as a real turbo fault. Slewing keeps the sandbox inside the physics the twin was built for. Rates are per second of *simulation* time, so 5× and 10× speed the climb up with everything else.

The demonstration that matters

- 1 Open **Sandbox** and switch to **Mode: manual sandbox**. The aircraft is picked up wherever the mission left it.
- 2 Set the rate to **5x** or **10x** so the climb completes in about a minute of real time.
- 3 Set throttle to **95 %** and altitude to **0 ft**. Let it settle.
- 4 Drag altitude to **30,000 ft** and watch.

ALTITUDE	MAP	POWER	HEALTH	DIAGNOSIS
0 ft	2.36 bar	187 HP	95.3 %	HEALTHY
10,025 ft	2.36 bar	184 HP	99.7 %	HEALTHY
20,050 ft	1.99 bar	141 HP	96.5 %	HEALTHY
30,000 ft	1.54 bar	110 HP	97.9 %	HEALTHY

Measured from the running twin across a continuous climb. Power derates 187 → 110 HP; health does not follow it down. Zero false diagnoses across the climb.

Every one of those readings would trip a fixed redline. That is the argument, in one gesture.

10 Recording, debrief and replay

What gets recorded

The flight data recorder starts with the server and records automatically — there is no button to arm it. Every broadcast frame is appended to disk in `data/flight_logs/<sortie-id>/` :

FILE	CONTENTS
meta.json	Sortie header: start time, duration, frame count, faults seen, minimum health, worst band
frames.jsonl	One serialised twin frame per line, in time order

Why JSONL

Frames are appended during flight with no rewrite, the log can be inspected with ordinary tools, and a sortie interrupted by a hard shutdown still reads back to its last complete line.

Debrief controls

Sortie dropdown	Lists recorded sorties, newest first, with duration and fault count. The one currently recording is marked.
Refresh	Rescans the log directory.
► Replay	Streams the selected sortie back through the live telemetry path.
■ Live	Returns to live telemetry. The simulation kept running underneath, so you rejoin a mission in progress.
Live / Replay n% badge	Shows the current source and, during replay, position through the sortie.

The post-flight report

Selecting a sortie produces a report answering what an operator asks after landing:

- Duration, frame count and altitude range flown.
- Health minimum and mean, and the worst band reached.

- **Time in each health band** — how long the engine spent NOMINAL, ADVISORY, CAUTION, WARNING and CRITICAL.
- **Time in each mission phase.**
- **Per-subsystem health minima** across the whole sortie.
- **Trigger-to-diagnosis latency** — the gap between the unsupervised trigger firing and the classifier committing to a name. This is measurable from the log without ground truth.
- **The diagnosis timeline** — each event with its class, onset time, duration, and the residual channels that drove it.

The timeline is debounced

Per-frame classification flickers, and at 20 Hz a single ambiguous frame would otherwise open and close a 0.05 s “event”. A class change is only accepted once it has persisted for a second — the same rolling-vote discipline the live annunciator uses, applied to the log.

Replay behaves exactly like live

Replay is a property of the *stream*, not of the twin. Recorded frames are pushed down the same transport the live twin uses, so the whole console — telemetry strips, health matrix, 3D view, evidence drawer — replays without modification. The playhead is advanced by the simulation clock rather than by consumer reads, so replay runs at true speed no matter how many clients are attached.

11 Demonstration scripts

Five minutes, unattended

Press ► **Auto demo** and let it run. Forty seconds, six phases, hands-free.

Best when you need to talk over it.

Five minutes, driven

- 1 **Watch it loiter.** High-altitude cruise. Every strip shows the measurement tracking the physics baseline. Point out that the absolute numbers look nothing like sea level, and nothing complains.
- 2 **Break the oil system.** Hit **Oil pressure loss**. Under three seconds later the trigger fires, the classifier names it, the sump goes red on the model, and the directive reads return to base.
- 3 **Break a sensor instead.** Clear, then hit **Probe defect (EGT3)**. EGT₃ falls toward ambient — the case a redline system aborts a healthy aircraft over. Health barely moves; the verdict is sensor probe defect, engine healthy.
- 4 **Ask it why.** Hit **Bearing 2X vib** and read the attribution panel, or click the crankcase in the model to open the inspector on the vibration channel.
- 5 **Fly the envelope.** Section 09 — climb to 30,000 ft at 5× and watch power derate while health holds.
- 6 **Land and debrief.** Open **Debrief**, pick the sortie, read the report, then replay it.

If you only have ninety seconds

Do step 3 and step 5. The probe defect shows what nothing else on the market does, and the climb shows why the whole architecture exists.

12 The HTTP and WebSocket API

The service listens on `127.0.0.1:8000`. Everything the dashboard does, you can do from code.

METHOD	ENDPOINT	DOES
GET	/	The ground station
GET	/api/state	Full twin state, once
POST	/api/fault/inject	Inject a fault
POST	/api/fault/clear	Back to healthy
POST	/api/flight/override	Manual throttle / altitude / OAT
POST	/api/sim/reset	Restart from $t = 0$
POST	/api/sim/speed	Time scale, clamped $0.2\times$ – $10\times$
GET	/api/sessions	Recorded sorties, newest first
GET	/api/sessions/{id}	Post-flight report
GET	/api/sessions/{id}/frames	Raw frames, paged
POST	/api/replay/start	Replay a sortie
POST	/api/replay/stop	Back to live
WS	/ws/telemetry	20 Hz state broadcast
GET	/api/stream	Same broadcast over SSE

Injecting a fault

```
curl -X POST http://127.0.0.1:8000/api/fault/inject \  
  -H "Content-Type: application/json" \  
  -d '{"fault_type": "TURBO_BOOST_DEFICIENCY",  
      "severity": 0.7,  
      "ramp_duration_s": 12.0}'
```

Severity is 0.0–1.0 and ramp is in seconds — `0` for an abrupt step. An unknown class returns HTTP 400.

Reading the stream

```
import asyncio, json, websockets

async def listen():
    async with websockets.connect("ws://127.0.0.1:8000/ws/telemetry") as ws:
        while True:
            s = json.loads(await ws.recv())
            ai = s["ai_prognostics"]
            print(f'{s["timestamp_s"]:7.1f}s   {s["altitude_ft"]:6.0f}ft   '
                  f'health {s["health"]["overall_health"]:5.1f}%   '
                  f'{ai["fault_class"]}')

asyncio.run(listen())
```

Each frame carries the flight state, every raw sensor channel, the model expectation for each, the filtered estimate, the residual vector, seven health scores and the AI block. The full schema is `digital_twin/twin_state.py` .

The twin runs whether or not anyone is watching

Simulation advances on a background task, so `/api/state` is populated with no browser attached and a second dashboard tab observes the same sortie rather than stepping it a second time.

13 Verifying the claims

Every number in this handbook comes out of a script in the repository.

```
python run_all_tests.py                # all nine suites, ~2.5 min

python test_simulation.py              # MVEM, sensors, flight profiles
python test_ai_layer.py               # estimator, autoencoder, classifier, RUL
python test_fault_pipeline_qa.py      # end-to-end detection per fault mode
python validation/benchmark.py        # the comparison table in section 01
python validation/ablation.py         # which choice is load-bearing
python validation/leakage_audit.py    # train/test separation integrity
python validation/stress_testing.py   # 5x noise, 30 s blackout, 35,000 ft
python tools/edge_benchmark_profiler.py # the latency budget
```

Every suite exits non-zero on failure, so a green run means all nine genuinely passed. The ablation is the interesting one if you are sceptical: it retrains four networks on your machine and prints numbers it computed, rather than numbers anyone recorded.

Rebuilding data and models

```
python data/generate_dataset.py        # 78 independent sorties
python train_and_export_models.py      # retrain, recalibrate threshold
```

Only one script rewrites the committed datasets

`data/generate_dataset.py` is it. The test suite generates into a temporary directory, because overwriting the CSVs mid-run would leave the shipped checkpoints scored against data they were never trained on — which is exactly the failure that once made the benchmark report the system losing to its own baseline.

14 Troubleshooting

SYMPTOM	CAUSE AND FIX
Health reads very low right after start	The warm-up did not complete. Press Reset and wait; the twin runs 15 s forward before the first frame is meaningful.
3D model is letterboxed or blank	The viewport was measured before the console settled. Resize the window once, or press ↑ Cover then Continue .
Charts and 3D view missing entirely	Vendored assets did not load. Confirm <code>dashboard/vendor/</code> is present; re-fetch with <code>python tools/fetch_vendor_assets.py</code> .
Badge says HTTP rather than WS	WebSocket blocked; the dashboard fell back to polling. Functionally identical, slightly coarser. Expected under the standalone server.
Sandbox sliders appear to do nothing	Mode is still auto mission . Toggle to manual. Also remember the slew rate — a large altitude change takes real time at 1×.
A fault is injected but nothing is diagnosed	Some faults ramp slowly by design. Give it the ramp duration plus a few seconds, or raise the simulation rate.
Diagnosis flickers between classes	Expected on marginal residuals. The annunciator votes over a rolling window before committing, so the displayed class is steadier than the per-frame one.
Debrief lists no sorties	The recorder starts with the server. Fly for a few seconds, then press Refresh .
Scrolling feels heavy on an old machine	Reduce the journey length: <code>--track</code> in <code>dashboard/theme.css</code> . Enabling the OS reduced-motion setting also disables parallax, blur and easing.

15 Known limitations

Stated here rather than left to be discovered. A reviewer who finds an unstated limitation discounts everything else in the submission.

1 — Everything is simulated

The engine, the instrumentation and the faults are models. No hardware-in-the-loop validation has been performed, and nothing here is certified for airworthiness or operational flight.

2 — The estimator is not an EKF

It is a linear Kalman filter with a complementary prediction step toward the physics model, with $F = I$ and $H = I$. Because F is identity, the covariance is a tuned noise-rejection parameter rather than a calibrated uncertainty. Anything needing meaningful state covariance would require a true EKF or UKF.

3 — The attribution is not SHAP

It is $\text{gradient} \times \text{input}$: a first-order local ranking without Shapley's local-accuracy or consistency guarantees. Chosen for the edge budget.

`shap.DeepExplainer` runs offline against the same network where exact values are wanted.

4 — The RUL interval is a trend interval

Constructed from the spread of the degradation fit, not from a distribution-free coverage guarantee. Conformal prediction is the honest upgrade.

5 — The sensor validator covers two archetypes

Thermocouple open circuit and oil transducer dropout. It isolates 93.3 % of held-out probe-defect frames, not 100 %. The ablation further shows the trained classifier already separates probe defects on its own, so the validator currently buys no measurable accuracy — its value is that it is a physics rule rather than a learned boundary, and so holds for probe failures absent from the training set.

6 — Per-frame inference degrades under extreme noise

At five times instrumentation sigma the classifier names a false mechanical fault on about one frame in six. The annunciator votes over a rolling window so this does not reach the operator, but it is a real weakness of per-frame inference on out-of-distribution noise.

7 — The MVEM fit figure is near-circular

The model interpolates the same altitude derating table it is then scored against, so the 0.00 HP RMSE is a consistency check on the interpolation,

not independent validation. Manifold pressure, which is not fitted to the table, tracks less exactly.

8 — The taxonomy is nine modes plus healthy

Real engines fail in compound and cascading ways this taxonomy does not name, though the unsupervised trigger will still flag them as anomalous — it just will not name them.

Where this goes next

Bench validation against a test-cell engine, recalibration on real data, an edge port to flight-representative compute, then secure telemetry and a requirements trace. The four stages and what each requires are chapter 21 of the cover journey.